

ECOSISTEMA DE AUTOMATIZACIÓN DE OPERACIONES

El Agente Autónomo de Control de Tareas, Alertas y Reportes Ejecutivos

Por Alberto Farah Blair | Code Ur Life

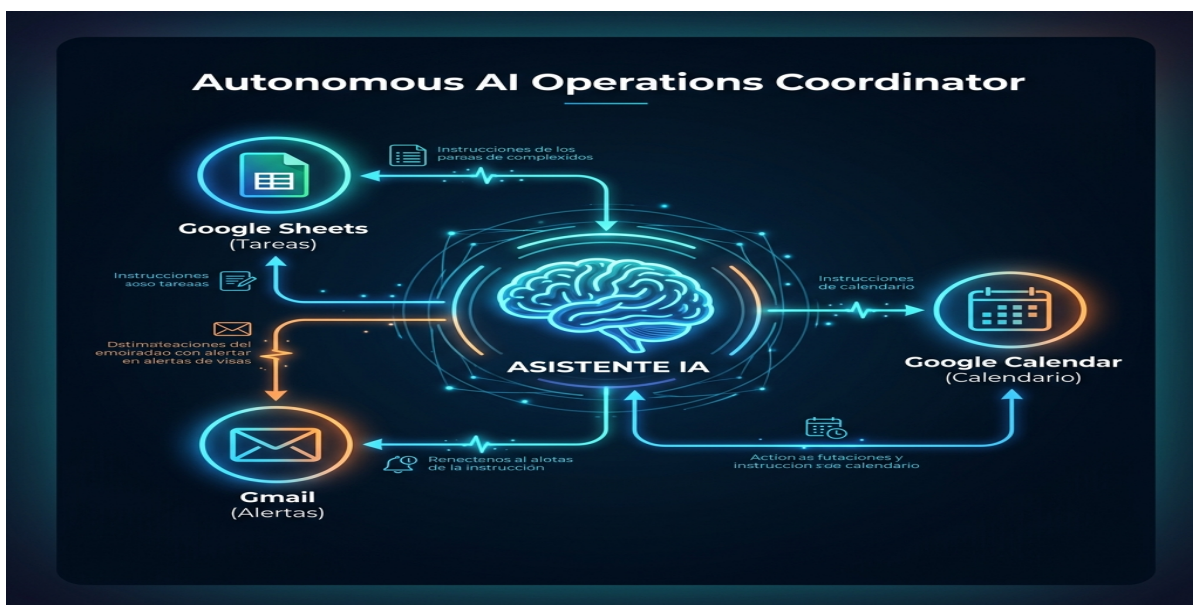
1. Introducción al Ecosistema

Perseguir a tu equipo para saber si avanzaron con sus tareas o si se cumplieron las fechas límite es uno de los mayores cuellos de botella en cualquier agencia o empresa de servicios. El micro-management consume horas de directores y gerentes, reduce la moral del equipo y, en el peor de los casos, provoca entregas atrasadas y clientes insatisfechos.

Este blueprint documenta una solución real implementada con éxito: un **Agente de Operaciones** que sincroniza Google Sheets (como base de datos de control), Gmail (como canal de alertas y reportes) y Google Calendar (para la agenda), operando en segundo plano de manera 100% automatizada.

2. Arquitectura del Flujo

A continuación se muestra el diagrama de integración y flujo de decisiones del Agente de IA:



3. Modelo de Datos de Google Sheets

Para que el agente de operaciones funcione de forma autónoma, requiere una estructura de datos clara. La base de datos se almacena en un documento de Google Sheets con dos pestañas fundamentales:

Pestaña 1: EQUIPO

Esta pestaña sirve como directorio. El agente mapea los nombres a sus correos electrónicos para enviar las alertas.

Nombre	Rol	Correo Electrónico
Alberto	Director	alberto@brandistry.com
Junior	Programador	junior@brandistry.com
QA	QA Specialist	qa@brandistry.com

Pestaña 2: TAREAS_OPS

El registro central de entregables. El agente escanea este listado para encontrar tareas pendientes y calcular los retrasos.

ID	Cliente	Tarea	Responsable	Prioridad	Estado	Fecha Límite
1	Cliente A	Configurar API	Junior	Alta	Pendiente	22/06/2026
2	Cliente B	Diseñar Reels	Anabel	Media	Hecho	24/06/2026

4. Guía de Construcción y Código (Parte 1)

A continuación se presenta el código funcional en Python estructurado en tres partes. Utiliza la librería oficial google-api-python-client. Asegúrate de generar tu archivo token.json mediante el flujo OAuth2 de Google Cloud.

Sección A: Configuración y Conexión de API

```
# CONFIGURACIÓN Y CONEXIÓN A LAS APIS DE GOOGLE
import os
import datetime
from googleapiclient.discovery import build
from google.oauth2.credentials import Credentials

SPREADSHEET_ID = 'TU_SPREADSHEET_ID_DE_GOOGLE_SHEETS'
EMAIL_MANAGER_1 = 'socio1@tuagencia.com'
EMAIL_MANAGER_2 = 'socio2@tuagencia.com'

def get_google_services():
    # Carga de credenciales y autorización de Sheets y Gmail
    creds = Credentials.from_authorized_user_file('token.json', [
        'https://www.googleapis.com/auth/spreadsheets',
        'https://www.googleapis.com/auth/gmail.send'
    ])
    sheets_service = build('sheets', 'v4', credentials=creds)
    gmail_service = build('gmail', 'v1', credentials=creds)
    return sheets_service, gmail_service

def send_email(gmail_service, to, subject, body):
    from email.mime.text import MIMEText
    import base64
    message = MIMEText(body)
    message['to'] = to
    message['subject'] = subject
    raw = base64.urlsafe_b64encode(message.as_bytes()).decode()
    try:
        gmail_service.users().messages().send(userId='me', body={'raw': raw}).execute()
        return True
    except Exception as e:
        print(f"Error al enviar correo a {to}: {e}")
        return False
```

4. Guía de Construcción y Código (Parte 2)

Sección B: Escaneo de Retrasos y Envío de Alertas a Desarrolladores

El agente realiza los cálculos de fechas y dispara recordatorios amables a los dueños de tareas:

```
def process_task_alerts(sheets_service, gmail_service):
    # 1. Obtener la lista del equipo y sus correos electrónicos
    team_data = sheets_service.spreadsheets().values().get(
        spreadsheetId=SPREADSHEET_ID, range="'EQUIPO'!A2:C50"
    ).execute().get('values', [])
    team_emails = {row[0].strip().lower(): row[2].strip() for row in team_data if len(row) >= 3}

    # 2. Leer las tareas operativas
    tasks_data = sheets_service.spreadsheets().values().get(
        spreadsheetId=SPREADSHEET_ID, range="'TAREAS_OPS'!A2:K2000"
    ).execute().get('values', [])

    today = datetime.date.today()
    for row in tasks_data:
        row = row + [''] * (11 - len(row))
        task_id, _, cliente, _, tarea_desc, responsable, _, estado, fecha_limite_str, _, _ = row
        if not task_id or estado.lower() in ('hecho', 'completado', 'anulado'):
            continue

        # Parsear fecha de entrega (esperado: dd/mm/aaaa)
        try:
            deadline = datetime.datetime.strptime(fecha_limite_str.strip(), '%d/%m/%Y').date()
        except ValueError:
            continue

        # Verificar si hay una demora de 2 o más días
        delay_days = (today - deadline).days
        if delay_days >= 2:
            dest_email = team_emails.get(responsable.strip().lower())
            if dest_email:
                subject = f"■■■ [Seguimiento] Tarea retrasada por {delay_days} días ({cliente})"
                body = f"Hola {responsable},\n\n" \
                    f"El asistente de operaciones detectó una demora de {delay_days} días en la tarea:\n" \
                    f"■ Tarea: {tarea_desc} (ID: {task_id})\n" \
                    f"■■■ Cliente: {cliente}\n" \
                    f"■ Fecha Límite: {fecha_limite_str}\n\n" \
                    f"Por favor, actualiza el estado en el sistema o avísanos si necesitas ayuda."
                send_email(gmail_service, dest_email, subject, body)
```

4. Guía de Construcción y Código (Parte 3)

Sección C: Generación de Reportes Consolidados y Orquestación Principal

Sección que agrupa todo para enviar un reporte ejecutivo al final de la jornada laboral:

```
def send_daily_executive_report(sheets_service, gmail_service):
    tasks_data = sheets_service.spreadsheets().values().get(
        spreadsheetId=SPREADSHEET_ID, range="'TAREAS_OPS'!A2:K2000"
    ).execute().get('values', [])

    today = datetime.date.today()
    retrasadas, activas = [], []

    for row in tasks_data:
        row = row + [''] * (11 - len(row))
        task_id, _, cliente, _, tarea, responsable, prioridad, estado, limite, _, _ = row
        if not task_id or estado.lower() in ('hecho', 'completado', 'anulado'):
            continue
        try:
            deadline = datetime.datetime.strptime(limite.strip(), '%d/%m/%Y').date()
            days = (today - deadline).days
            if days >= 2:
                retrasadas.append(f"- [{cliente}] {tarea} - {responsable} (Demora: {days} días)")
            else:
                activas.append(f"- [{cliente}] {tarea} ({estado}) - {responsable}")
        except ValueError:
            activas.append(f"- [{cliente}] {tarea} ({estado}) - {responsable}")

    # Redactar correo consolidado
    body = f"■ REPORTE DIARIO DE OPERACIONES - {today.strftime('%d/%m/%Y')}\n"
    body += "="*50 + "\n\n"
    body += f"■ TAREAS CON RETRASO (2+ DÍAS): {len(retrasadas)}\n"
    body += "\n".join(retrasadas) if retrasadas else "Sin tareas retrasadas."
    body += f"\n\n■ TAREAS EN CURSO: {len(activas)}\n"
    body += "\n".join(activas) if activas else "Sin tareas en curso."

    for manager in [EMAIL_MANAGER_1, EMAIL_MANAGER_2]:
        send_email(gmail_service, manager, "■ Reporte Operativo Diario", body)

if __name__ == '__main__':
    sheets, gmail = get_google_services()
    process_task_alerts(sheets, gmail)
    send_daily_executive_report(sheets, gmail)
```

5. Instrucciones para tu Asistente de IA (Claude Code / Cursor)

Si utilizas un asistente de código basado en Inteligencia Artificial (como **Claude Code**, **Cursor**, **ChatGPT** o **Copilot**), no necesitas escribir el código a mano. Puedes entregarle este PDF completo y alimentarle el siguiente prompt listo para usar.

El asistente de IA leerá la arquitectura, las tablas de datos y estructurará el proyecto local automáticamente.

Actúa como un Ingeniero de Automatización Senior. Lee este documento completo e implementa el Agente de Operaciones en Python.

- 1. Instala las dependencias necesarias de Google Cloud en un entorno virtual:
`pip install google-api-python-client google-auth-http2 google-auth-oauthlib`*
- 2. Crea un archivo 'config.json' para almacenar el SPREADSHEET_ID y los correos de los directores.*
- 3. Escribe el script de Python estructurado e implementa las funciones de conexión, el escaneo de tareas retrasadas y el envío de correos.*
- 4. Escribe un script de prueba 'test_agent.py' que simule la carga de tareas de Sheets y el envío de Gmail usando logs de texto, print y logging.*